

Al-Furqan (The Criterion) — Architecture Document

Version: 1.0

Date: 2026-03-19

Author: بتوجيه من محمد الأشماوي ومحمود السمان (Arif AI) عارف

Status: Draft

Related: PRD v1.0

1. Architecture Overview

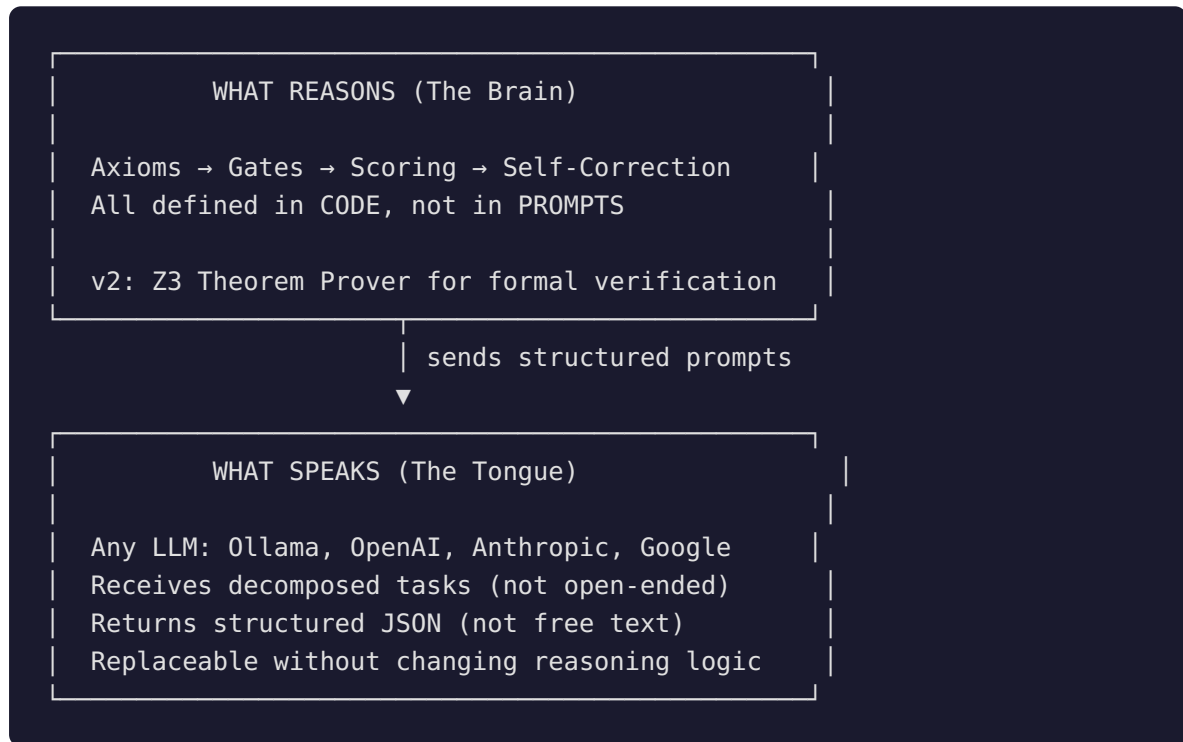
1.1 Design Philosophy

Al-Furqan follows a strict **separation of concerns** principle derived from an analogy with the human judicial system:

Judicial Concept	System Component	Role
The Constitution	Axioms & Proofs	Immutable foundation — never modified at runtime
Case Law	Verdict Store	Growing body of precedent — informs future reasoning
Legal Principles	Reasoning Patterns	Emergent from accumulated rulings
The Appeals Court	Human Review	Corrective, not generative
Procedural Safeguards	The 4 Gates	Structural, enforced on every evaluation
The Tongue/ Advocate	LLM	Provides language capability, NOT truth
Formal Verification	Symbolic Engine (v2)	Mathematical proof of gate compliance

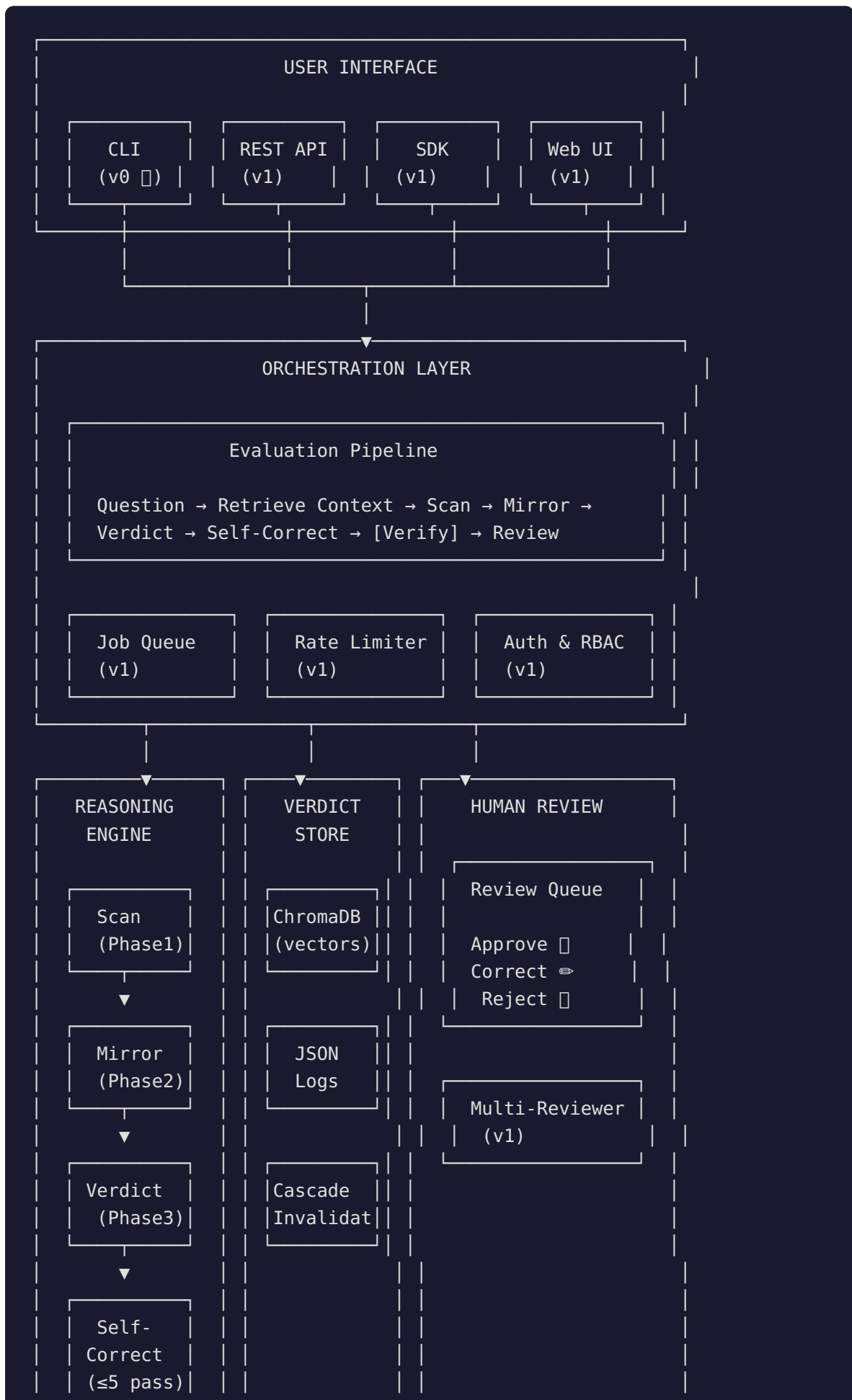
1.2 Core Principle: LLM as Tongue, Not Brain

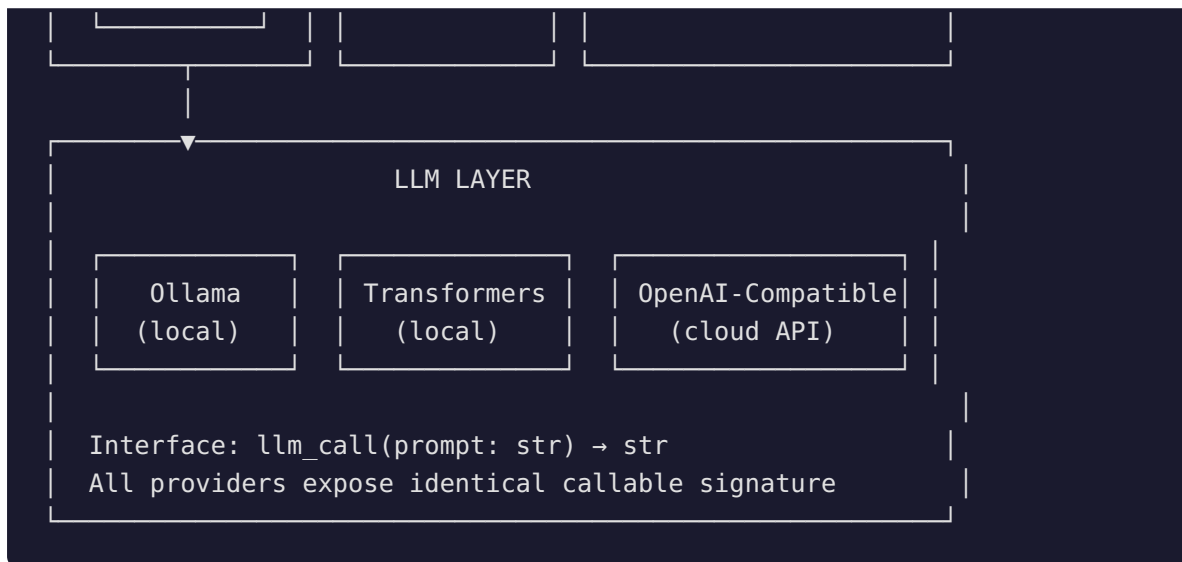
The single most important architectural decision: **the LLM does not reason**. It translates.



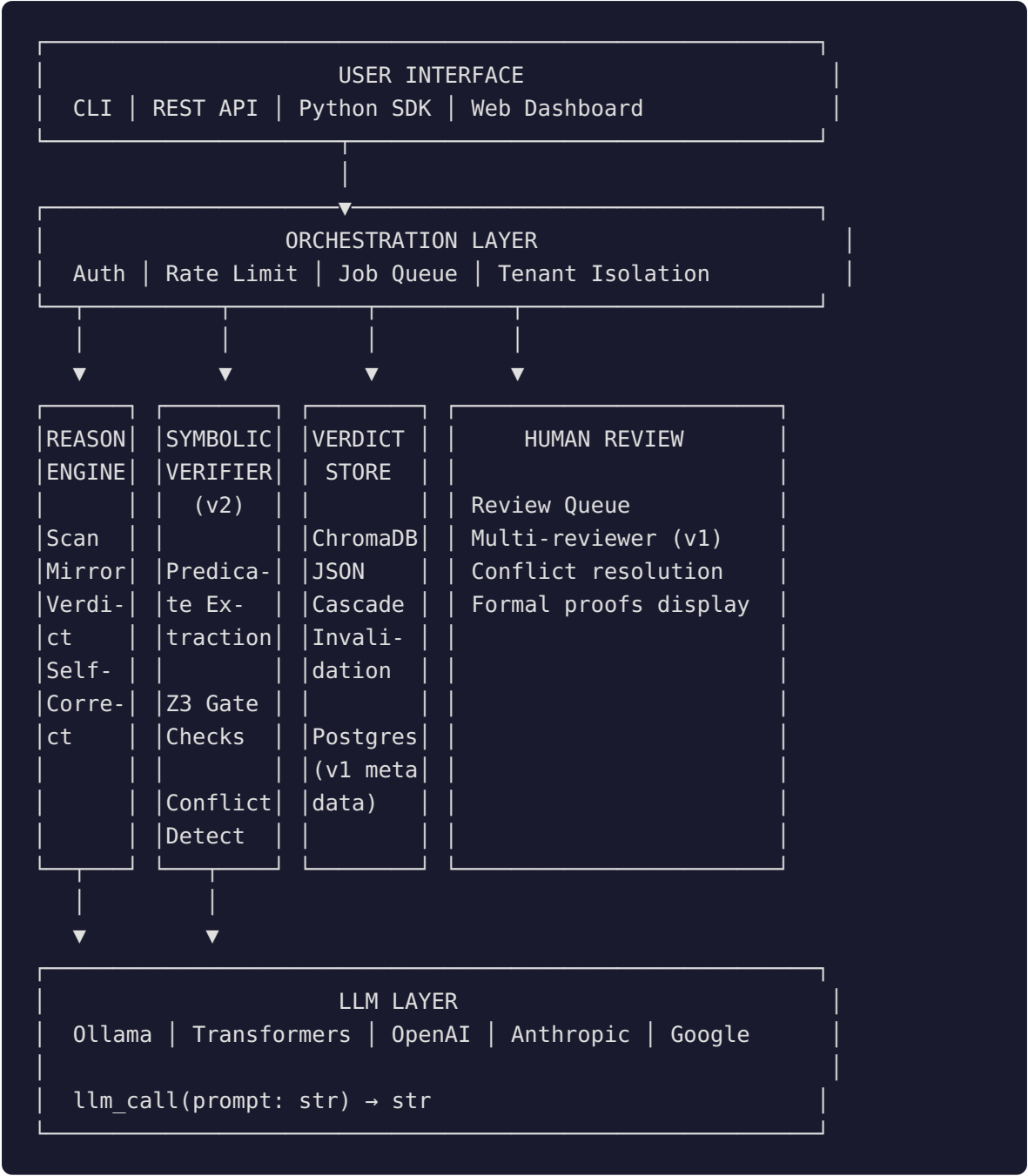
2. System Architecture

2.1 High-Level Architecture (Current: v0)





2.2 Target Architecture (v2: With Symbolic Verification)



3. Component Deep Dives

3.1 Reasoning Engine (reasoning_engine.py)

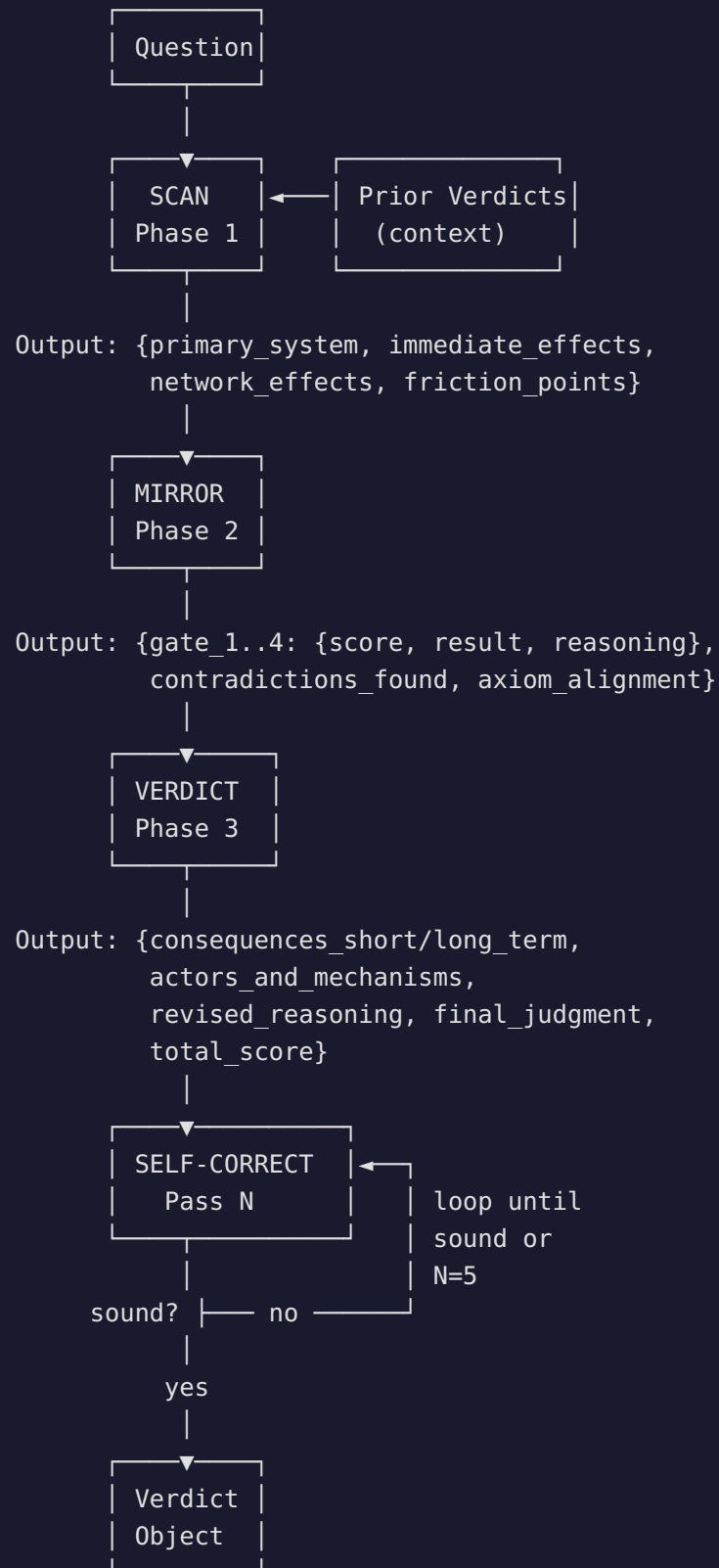
Responsibility: Execute the 3-phase evaluation pipeline and self-correction loop.

Current Implementation:

- ReasoningEngine class with single dependency: llm_call: Callable[[str], str]

- 4 prompt builders: `build_scan_prompt` , `build_mirror_prompt` ,
`build_verdict_prompt` , `build_correction_prompt`
- Framework constants: `FRAMEWORK_PREAMBLE` , `AXIOMS` , `GATE_DEFINITIONS` ,
`SCORING_RULES`
- Self-correction: up to `MAX_CORRECTION_PASSES` (default 5) iterations

Data Flow:



Key Design Decisions:

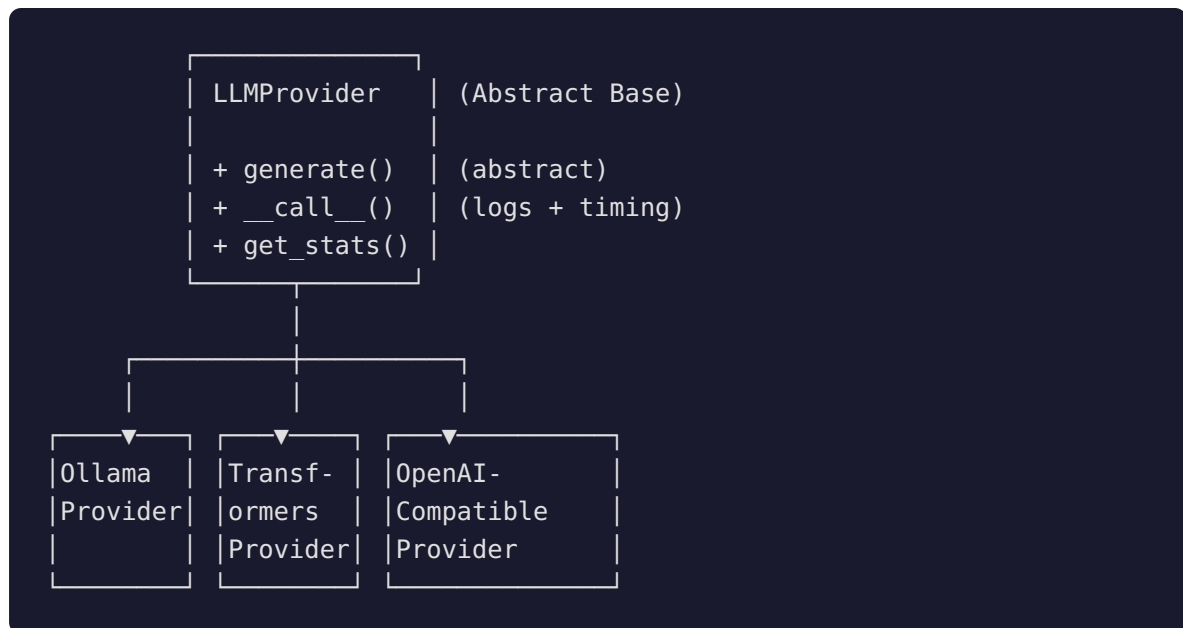
- Each phase gets its **own prompt** with full axiom context (no shortcutting)
- JSON structured output enforced at every step

- Phases are **sequential** — each uses output of previous as input
- Self-correction is **adversarial** — explicitly asks "find contradictions in this verdict"

3.2 LLM Layer (`llm_layer.py`)

Responsibility: Provide a unified, model-agnostic interface for text generation.

Architecture Pattern: Strategy Pattern



Interface Contract:

```
llm_call(prompt: str) → str
```

This is the **only** interface the reasoning engine depends on. Any model that can take text in and produce text out can be plugged in with zero changes to the reasoning logic.

Configuration:

```

llm:
  provider: ollama          # ollama | transformers | openai_compatible
  model_name: mistral      # model identifier
  base_url: "http://localhost:11434"
  temperature: 0.1         # low for deterministic reasoning
  max_tokens: 4096
  top_p: 0.9
  repeat_penalty: 1.1
  
```

3.3 Verdict Store (`verdict_store.py`)

Responsibility: Persistent storage with semantic retrieval — the system's "memory" and "case law."

Dual Storage Architecture:



Verdict Lifecycle:

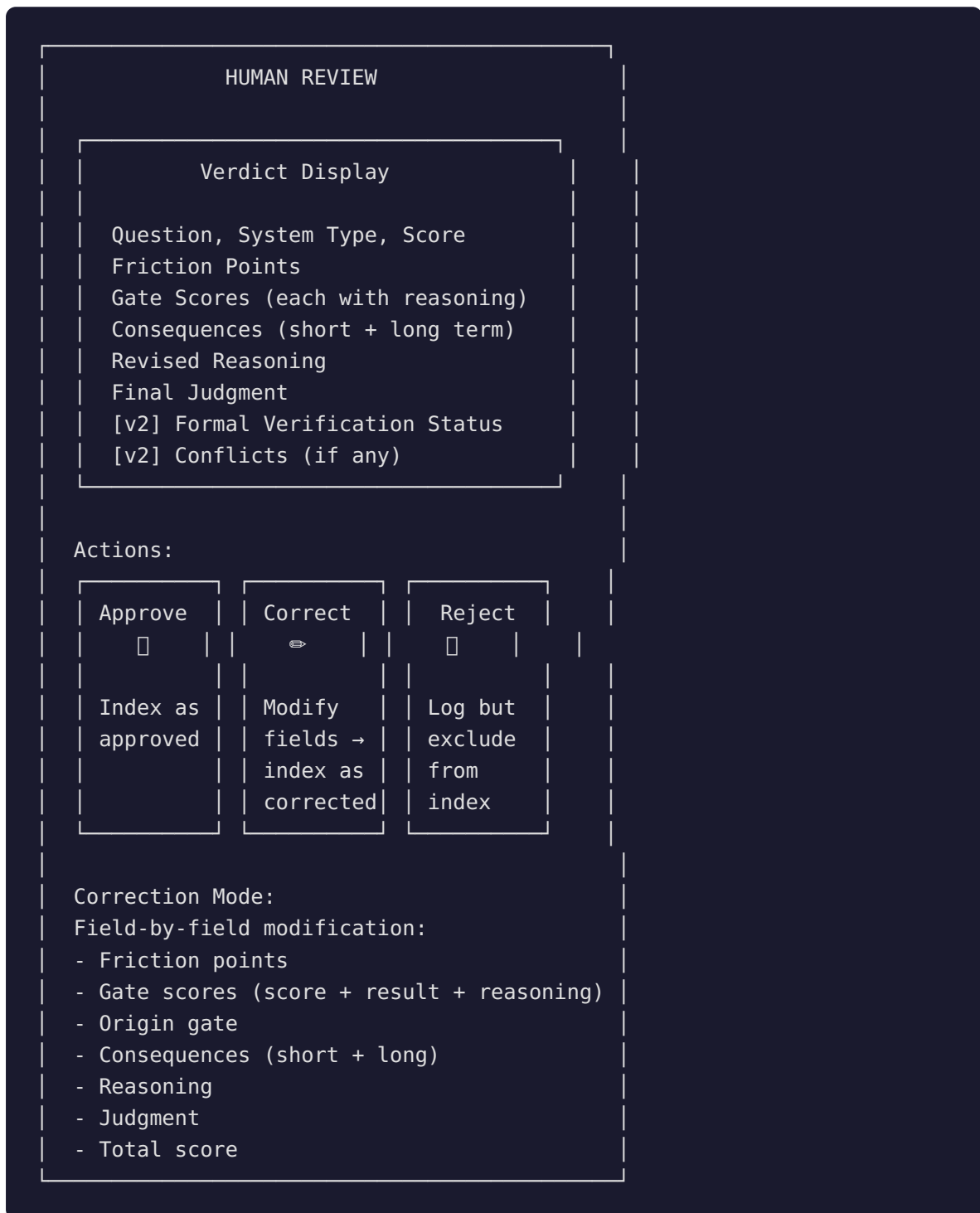


3.4 Human Review (`human_review.py`)

Responsibility: The "appeals court" — calibrates the system through human judgment.

Design Principle: The human reviewer is **corrective, not generative**. They don't produce verdicts — they validate, adjust, or reject machine-generated ones.

Review Actions:



Maturity Model:

Early Stage (0-100 verdicts)		Growth Stage (100-1000)		Maturity Stage (1000+)
Human corrects frequently	→	Corrections decrease as precedent grows	→	Human shifts from corrector to auditor
Verdict store is sparse	→	Rich context for new evals	→	System handles routine cases
Every verdict needs review	→	High-score auto-approved	→	Only novel cases need human review

3.5 Symbolic Verification Engine (v2: `symbolic/`)

Responsibility: Formally verify gate results using mathematical logic, independent of LLM output.

Architecture:



Z3 Verification Logic:

For each gate:

1. Encode axioms as Z3 constraints
2. Encode extracted predicates as Z3 assertions
3. Assert the NEGATION of the gate's survival condition
4. Run Z3 solver:
 - If UNSAT → the negation is impossible
→ gate SURVIVES (formally proven)
 - If SAT → a counterexample exists
→ gate FAILS (with counterexample)

4. Data Architecture

4.1 Data Models

Verdict (Core Entity):

```

Verdict
├─ id: string (verdict_{timestamp})
├─ question: string
├─ primary_system: enum (economic|social|spiritual|
                        political|legal|technological|
                        environmental|mixed)
├─ friction_points: string[]
├─ gate_scores: GateScore[]
│   └─ name: string
│   └─ score: int (0-100)
│   └─ result: enum (Survive|Fail)
│   └─ reasoning: string
├─ origin_gate: enum (Survive|Fail)
├─ consequences_short_term: string[]
├─ consequences_long_term: string[]
├─ revised_reasoning: string
├─ final_judgment: string
├─ total_score: int
├─ passes: int (self-correction passes)
├─ timestamp: float
├─ status: enum (pending|approved|corrected|
                rejected|superseded|needs_review)
├─ [v2 additions]
├─ symbolic_verification: object|null
│   └─ formally_verified: bool
│   └─ symbolic_gate_results: SymbolicGateResult[]
│   └─ conflicts: Conflict[]
│   └─ trust_level: enum (FORMAL|REQUIRES_REVIEW|
                        NOT_VERIFIED)
├─ [v1 additions]
├─ reviewer_id: string|null
├─ review_timestamp: float|null
├─ review_notes: string|null
├─ parent_verdict_id: string|null (for corrections)
└─ influenced_by: string[] (verdict IDs used as context)

```

4.2 Storage Architecture

Current (v0):

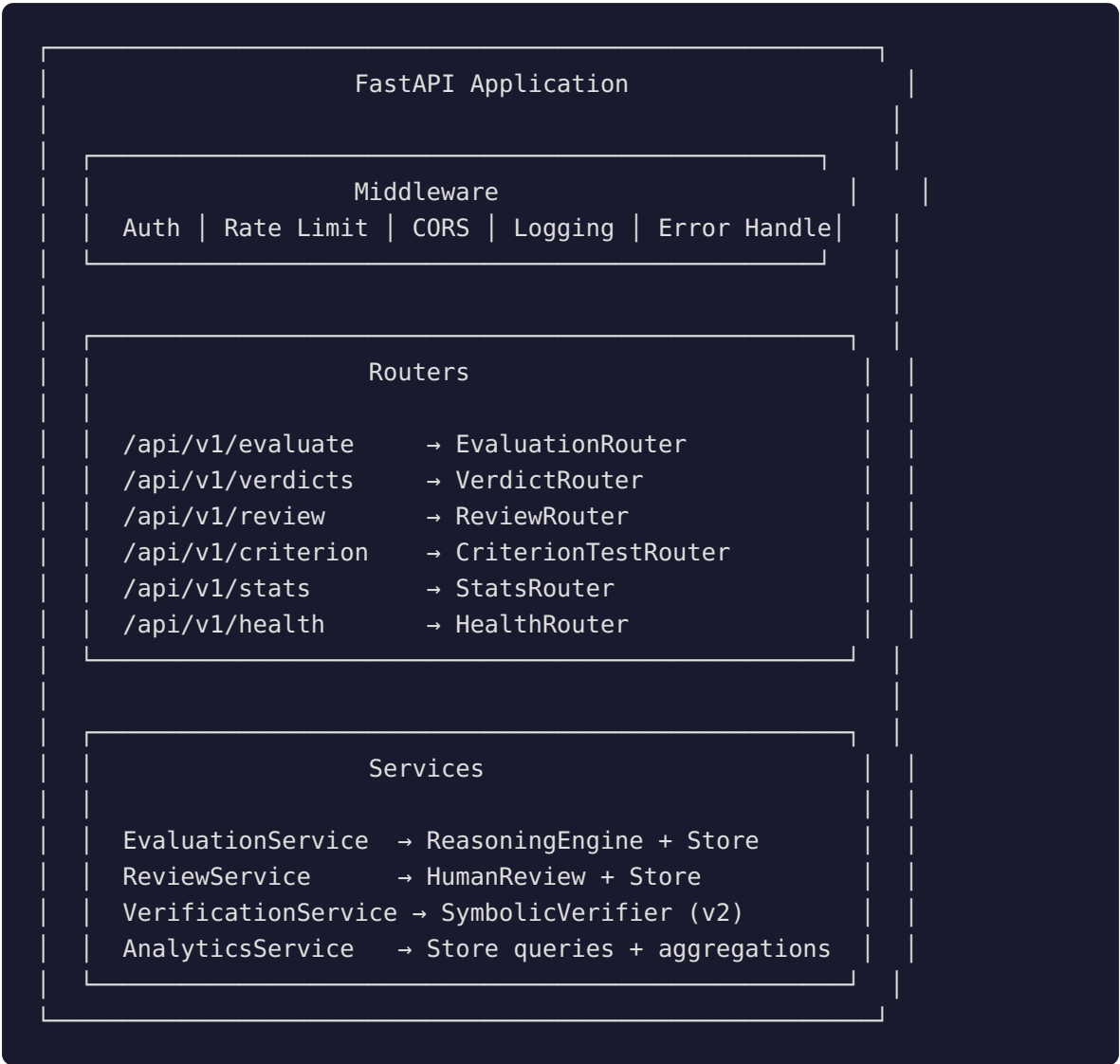


Target (v1):



5. API Architecture (v1)

5.1 API Layer Design



5.2 API Endpoints

```
POST    /api/v1/evaluate
        Body: {question, context?, options?}
        Response: {verdict_id, status: "processing"}
        → Async: evaluation runs in background

GET     /api/v1/evaluate/{verdict_id}
        Response: {verdict} or {status: "processing", progress}

POST    /api/v1/criterion-test
        Body: {framework_name, framework_description}
        Response: {test_id, status: "processing"}
        → Runs full Criterion Test

GET     /api/v1/verdicts
        Query: ?status=approved&system=social&limit=20
        Response: {verdicts[], total, page}

GET     /api/v1/verdicts/{id}
        Response: {full verdict with all fields}

POST    /api/v1/verdicts/{id}/review
        Body: {action: "approve"|"correct"|"reject",
               corrections?: {...}, notes?: string}

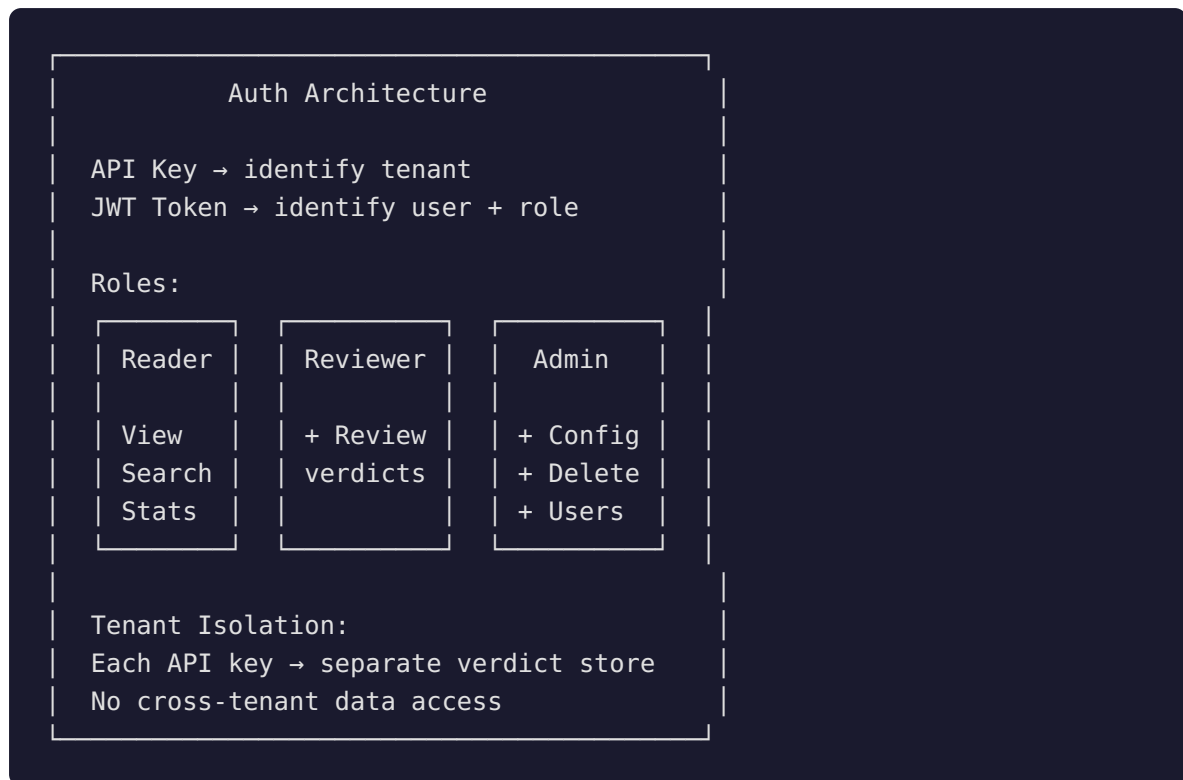
DELETE  /api/v1/verdicts/{id}
        → Triggers cascade invalidation

GET     /api/v1/verdicts/search
        Query: ?q=secular+humanism&limit=5
        Response: {results[] with relevance scores}

GET     /api/v1/stats
        Response: {total_verdicts, by_status, by_system,
                  accuracy_trend, avg_score}

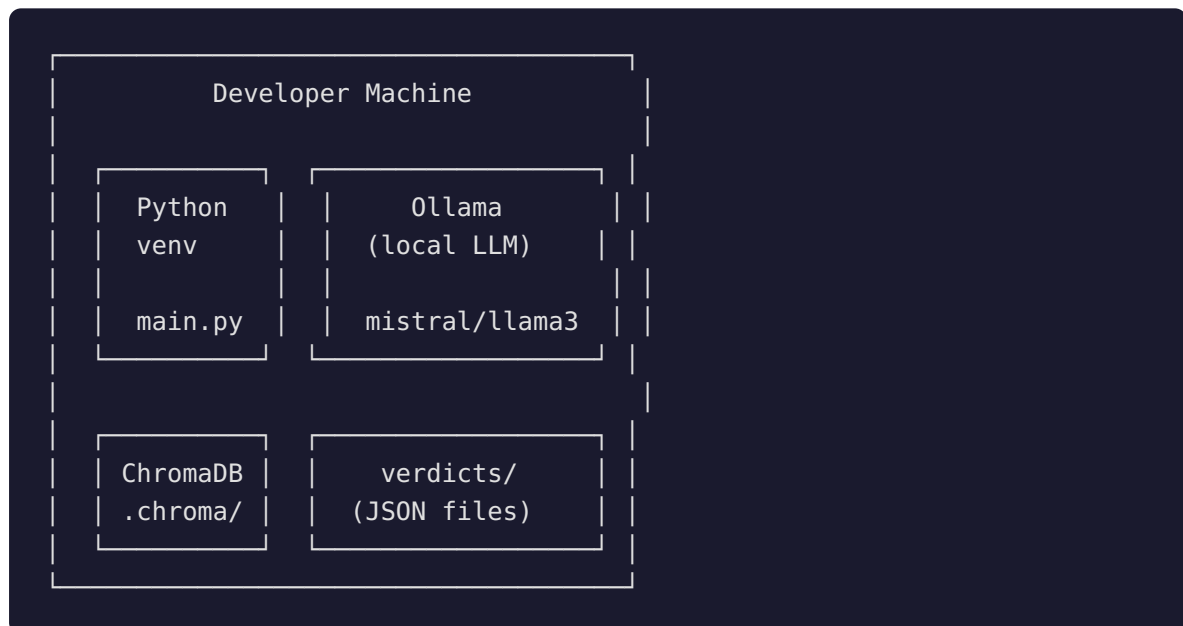
GET     /api/v1/health
        Response: {status, llm_status, store_status,
                  symbolic_status}
```

5.3 Authentication & Authorization

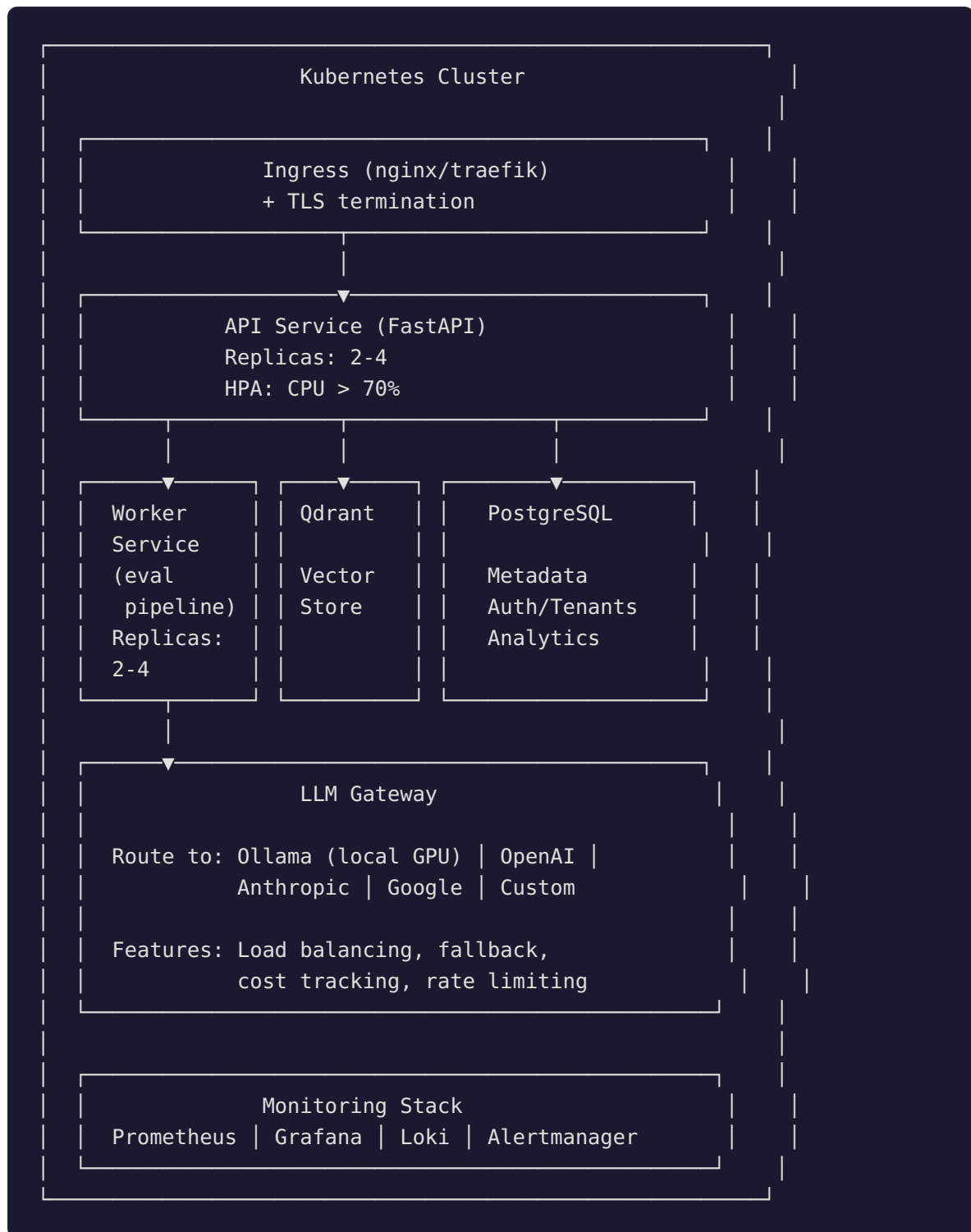


6. Deployment Architecture

6.1 Development (Local)



6.2 Production (Cloud/Self-Hosted)



6.3 Docker Compose (Intermediate)

```
# docker-compose.yml
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      - LLM_PROVIDER=openai_compatible
      - LLM_BASE_URL=http://ollama:11434
    depends_on: [ollama, qdrant, postgres]

  worker:
    build: .
    command: celery -A worker worker
    depends_on: [redis, ollama]

  ollama:
    image: ollama/ollama
    volumes: ["ollama_data:/root/.ollama"]
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]

  qdrant:
    image: qdrant/qdrant
    volumes: ["qdrant_data:/qdrant/storage"]

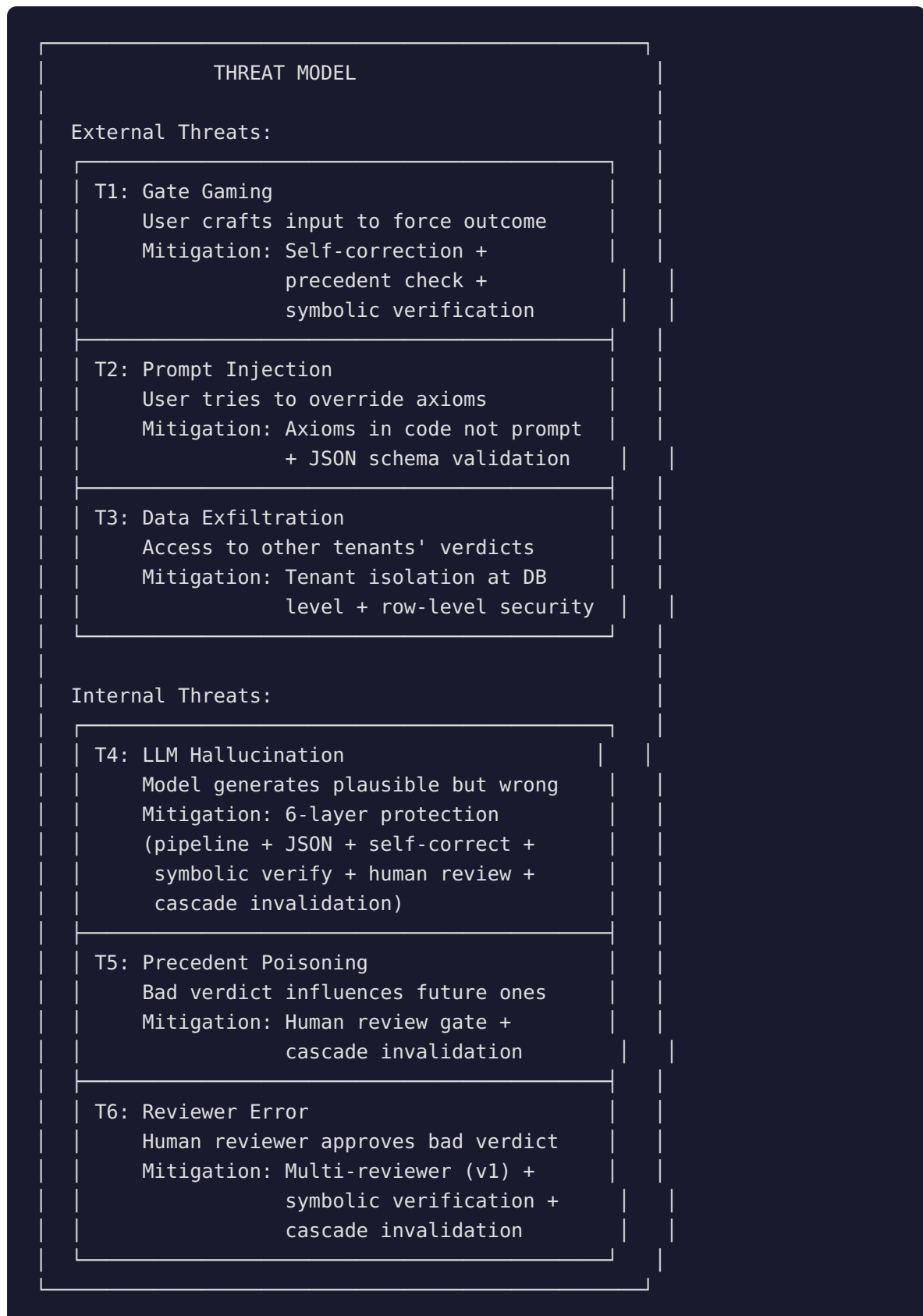
  postgres:
    image: postgres:16
    volumes: ["pg_data:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine

  dashboard:
    build: ./frontend
    ports: ["3000:3000"]
```

7. Security Architecture

7.1 Threat Model



7.2 Data Protection

DATA PROTECTION LAYERS

Transport:

TLS 1.3 (all connections)

Storage:

AES-256-GCM at rest

Access:

RBAC + tenant isolation

Audit:

All actions logged with
user ID + timestamp

Backup:

Daily automated + on-demand

Retention:

Configurable per tenant

Deletion:

GDPR-compliant cascade

8. Performance & Scalability

8.1 Performance Targets

Operation	Target Latency	Bottleneck
Full evaluation (cloud LLM)	< 60s	LLM inference (6-8 calls)
Full evaluation (local 7B)	< 120s	Model inference
Semantic search	< 500ms	Vector DB query
API endpoints (non-eval)	< 200ms	DB query
Symbolic verification	< 5s	Z3 solver
Dashboard page load	< 2s	Frontend + API

8.2 Scalability Strategy

SCALABILITY APPROACH

Horizontal:

- └─ API replicas (stateless, behind LB)
- └─ Worker replicas (eval pipeline)
- └─ Read replicas (Qdrant/PostgreSQL)

Vertical:

- └─ GPU for local LLM inference
- └─ Memory for vector store

Caching:

- └─ Redis: recent verdicts, session data
- └─ CDN: dashboard static assets
- └─ LLM response cache (same question = same predicates, skip re-extraction)

Async:

- └─ Evaluations run as background tasks
- └─ Webhook notification on completion
- └─ Polling endpoint for status

9. Evolution Roadmap

v0 (Current)	v1 (Foundation)	v2 (Symbolic)
CLI only	+ REST API	+ Z3 Verification
Local LLM	+ Cloud LLMs	+ Formal Proofs
ChromaDB	+ PostgreSQL	+ Conflict Detection
Single reviewer	+ Multi-reviewer	+ Trust Levels
JSON files	+ Dashboard	+ Proof Display
No auth	+ API keys + RBAC	+ Audit Trail
No tests	+ Benchmark suite	+ Symbolic Tests
	+ Docker deployment	+ K8s ready
v3 (Scale)	v4 (Ecosystem)	
+ Multi-tenant	+ Python SDK	
+ Enterprise SLA	+ Plugin system	
+ LLM gateway	+ API marketplace	
+ Monitoring stack	+ Community verdicts	
+ Auto-scaling	+ Training/cert	
+ Advanced analytics	+ Mobile app	

10. Decision Log

Decision	Rationale	Alternatives Considered
LLM as tongue, not brain	Ensures reasoning logic survives model changes; makes system LLM-agnostic	Fine-tuning a model (expensive, not transferable)
ChromaDB for vector store	Simple, local, Python-native; good for v0	Qdrant (planned for v1), Pinecone (cloud-only), Weaviate
JSON for verdict storage	Human-readable, git-friendly, simple	Database-only (less auditable)
Self-correction loop (max 5)	Catches contradictions without infinite loops	Single pass (misses errors), unlimited (expensive)
Structured JSON output	Forces specificity, enables validation	Free text (harder to parse, more hallucination)
Human-in-the-loop mandatory	System is calibrative, not autonomous; builds trust	Fully automated (less trustworthy)
Z3 for symbolic verification (v2)	Industry standard theorem prover; Python bindings	Prolog (less formal), custom solver (reinventing wheel)
FastAPI for API layer (v1)	Async support, auto-docs, Python native	Django (heavier), Flask (less features), Go (different ecosystem)

This architecture document is a living document. Last updated: 2026-03-19